Replicating Min-K% Prob

Project update: detecting pretraining data from large language models

Ashikur Rahman 48483818

9 October 2026 github.com/Ashik-here/COMP8240-MinK-Replication

COMP8240 Applications of Data Science, Macquarie University

The question and the paper

Can an outsider tell whether a text was in a model's training data?

Copyright, benchmark contamination, privacy: all need this "membership inference".

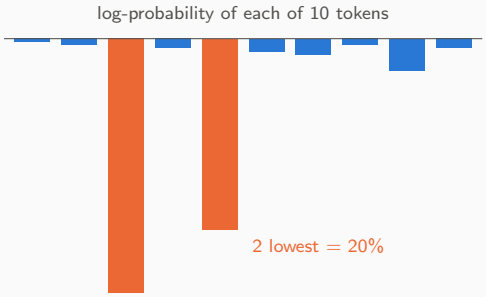
Shi et al., ICLR 2024

- ICLR: CORE **A***, Google Scholar top-10 venue
- Code: `swj0419/detect-pretrain-code`
  Apache-2.0, pinned at commit `a5048f9`
- Data: WikiMIA on Hugging Face (MIT)

Why this paper

- runs on a free Colab T4
- ∼250 lines: I can explain all of it
- claim is **contested**
  Duan et al. 2024; Das et al. 2024

The method: Min-K% Prob



- Average of *all* tokens: -1.7
- Average of the **lowest 20%**: $(-8.0 - 6.0)/2 = \mathbf{-7.0}$
- Unseen text has a few very surprising words; training removes them
- Higher score $\Rightarrow$ predict **member**
- One forward pass, no reference model

Original data: WikiMIA

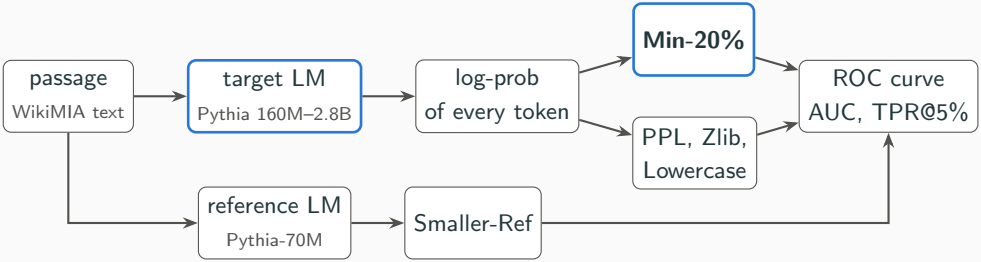
Labels come from dates, not from the model's (secret) training set

- **Members**: Wikipedia event articles created *before 2017*
- **Non-members**: event articles created *after 1 Jan 2023*, after every model's cut-off
- Same passages cut to 32 / 64 / 128 / 256 words, balanced 50/50

Length (words)	32	64	128	256
Passages	776	542	250	82

Metrics: AUC (chance = 0.5) and TPR at 5% false-positive rate.

System architecture



`run.py` scores each passage → `eval.py` writes `auc.txt` + ROC plot.
I import the original functions unmodified and call them from a notebook, so per-passage scores are kept (the original script discards them).

Replication on original data: status

Pythia-2.8B, mean of 4 lengths

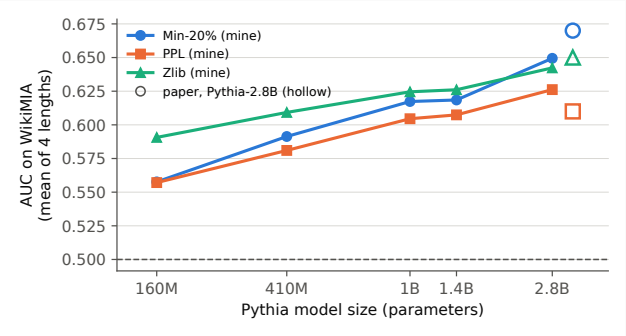
Method	Mine	Paper
PPL	0.63	0.61
Zlib	0.64	0.65
Lowercase	0.60	0.59
Smaller Ref	0.61	0.60
Min-20%	**0.65**	**0.67**

All five within 0.02 of the paper.

size $0.56 \rightarrow 0.65$, length $0.62 \rightarrow 0.70$

Min-K% leads Zlib by <0.01: inside the CIs.

Colab T4, original code at a5048f9, transformers 5.18, fp16.



Scope: the paper's 20B–66B models exceed a free T4, so its size claim is tested across five Pythia sizes instead.

What replication needed: five traps in the original repo

1. No `requirements.txt` ⇒ I pin and record versions myself
2. Imports `openai` always; GPT-3 rows use `text-davinci-003`, **retired** ⇒ those rows can never be reproduced
3. Score stored negated, then negated again in `eval.py` ⇒ a re-implementation can silently flip the ROC
4. A committed `out/` folder holds the *authors'* LLaMA-13B results ⇒ easy to read theirs instead of yours
5. `auc.txt` says "TPR@0.1%FPR" but the code computes TPR@**5%**

Also: Table 1 does not say which length it reports, so I report all four plus the mean. Independent 30-line re-implementation matches the repo to 3 decimals.

New data 1: existing datasets the paper did not use

arXivTection (DE-COP, ICML 2024) **done**

- 50 arXiv papers, **1,549** passages
- label: published before 2022 vs. in 2023
- shipped as multiple choice ⇒ adapter keeps the verbatim option
- caveat: labels suit 2022+ models, not Pythia

MIMIR (COLM 2024) **obtained**

- Pile *train* vs. *test*: same time, same sources
- built for Pythia: my primary target
- **gated** since my proposal ⇒ access granted today; Wikipedia split, 1,000 + 1,000 passages
- kept local: gated data is not redistributed

BookTection: public, same adapter next

New data 2: WikiMIA-2026, built with the paper's own recipe

Pipeline (`scripts/build_wikimia2026.py`)

1. MediaWiki API: event categories of 2026 vs. 2012–15
2. creation date = first revision; keep $\geq$ 1 Jan 2026 / $<$ 2017
3. drop "List of…", "Timeline of…"; regex clean
4. cut to 32/64/128/256 words, balance 50/50
5. JSONL in the original schema $\Rightarrow$ `run.py` unmodified

Change from WikiMIA: *same* 16 topic types on both sides.

First attack result (original code, Pythia-2.8B): Min-K% **0.68** (WikiMIA 0.65), PPL 0.66, Zlib 0.63; size $0.56 \rightarrow 0.64 \rightarrow 0.68$

$\Rightarrow$ **the claim transfers to fresh data**

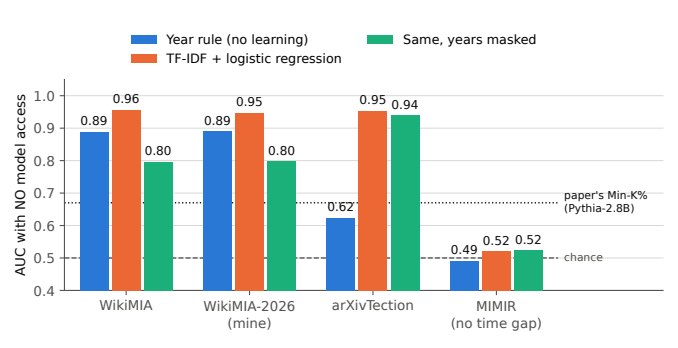
Built 2026-10-09

Length	Passages
32	1052
64	686
128	268
256	72

347 of 947 "2026" pages were created *before* 2026 and rejected. Each passage keeps page id, revision id, creation time.

Annotation: membership labels come *automatically* from creation dates. Human + LLM-judge labels: planned for the recency audit (ND-2).

Progress: does the benchmark leak its own labels?



WikiMIA bars: mean over the 4 lengths; 5-fold CV inside a pipeline (arXivTection: folds grouped by paper).

AUC of a classifier that **never sees the model**:

- WikiMIA **0.96** (Min-K%: 0.67)
- years masked 0.80 ("will be" vs. "was")
- my WikiMIA-2026 **0.95** (topics matched)
- arXivTection 0.95 (0.997 if folds leak papers)
- **control** MIMIR **0.52** $\approx$ chance

$\Rightarrow$ the leak is **time**, not memorisation.

Next steps

1. **Min-K% on MIMIR**, where the blind baseline is at chance: does the attack still beat 0.5?
2. **ND-2: a shift-controlled split**: members and non-members from the same time window; blind AUC must be ≈ 0.5 *before* any attack is trusted
3. Which passages does Min-K% get right: the same ones the blind classifier finds easy?
4. LLM-as-judge recency check (both A/B orders) + 200 human labels, Cohen's / Fleiss' κ; final report in LaTeX with the GitHub repo

So far: the method reproduces on old *and* new data;
the open question is whether it detects *memorisation* or *time*.

All code, data, results and run logs:
`github.com/Ashik-here/COMP8240-MinK-Replication`